



Departamento de Informática

Bacharelado em Ciência da Computação

CI1212 Arquitetura de Computadores

Memória Cache

PE-2020
Prof. Eduardo Todt
Versão 202008181804



Memória cache

Objetivos

Entender principais modelos de memória cache e seu impacto no desempenho



Tipos de falta na cache (3C)

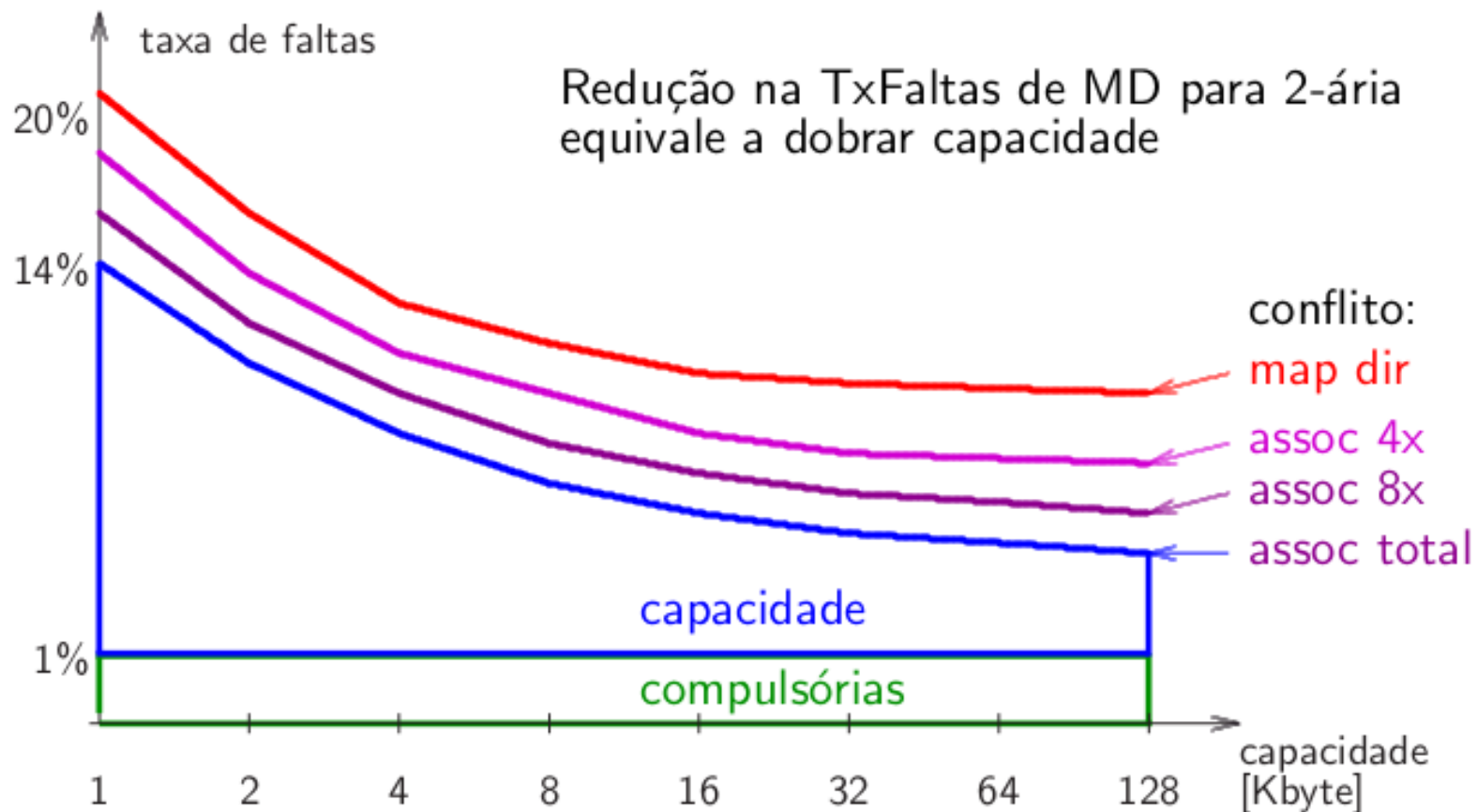
Falta por conflito

Falta por capacidade

Falta compulsória

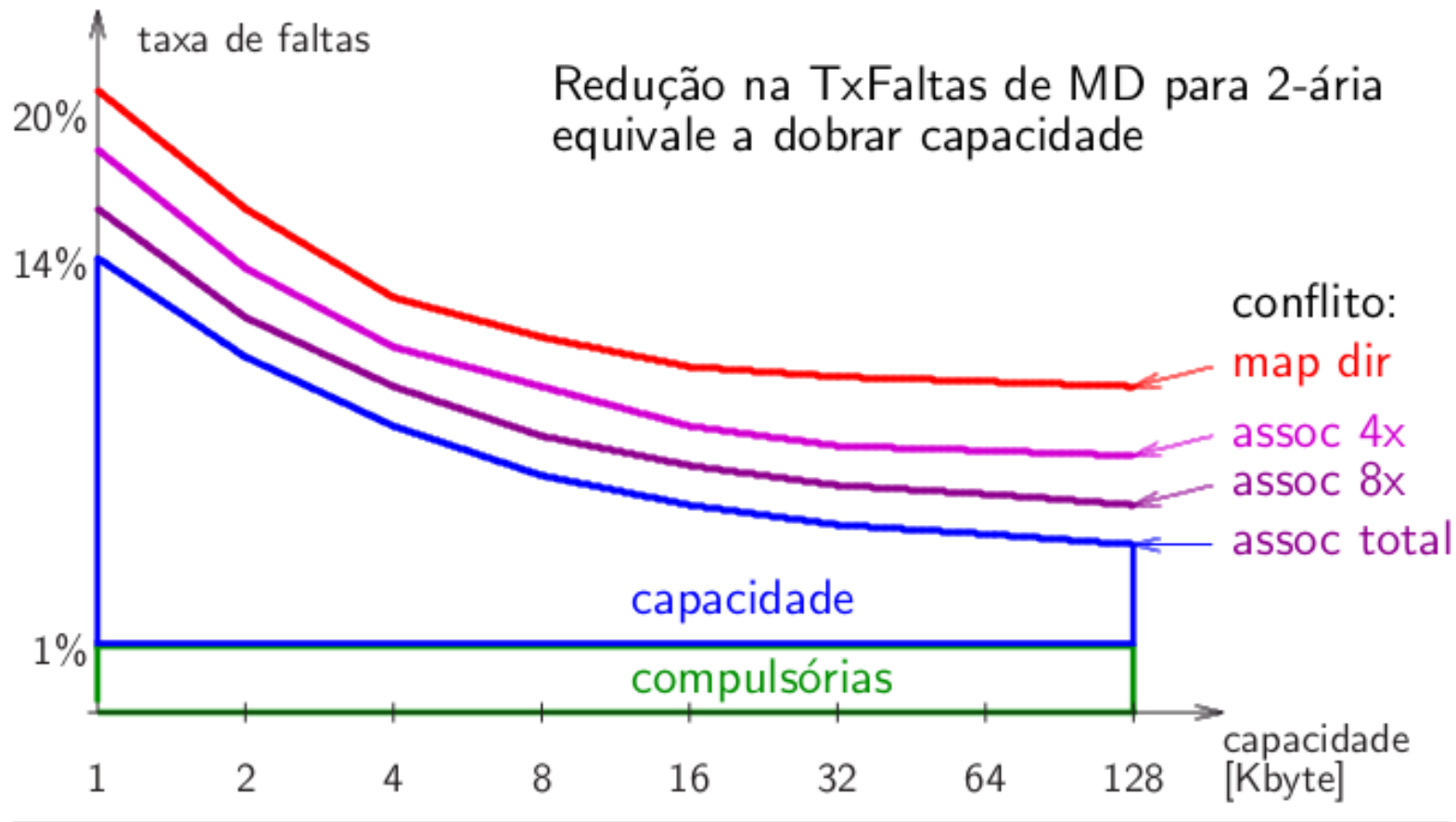
Falta por conflito

problema: N blocos da mesmo índice (cor) em uso simultâneo,
mas cache só acomoda $M < N$ blocos



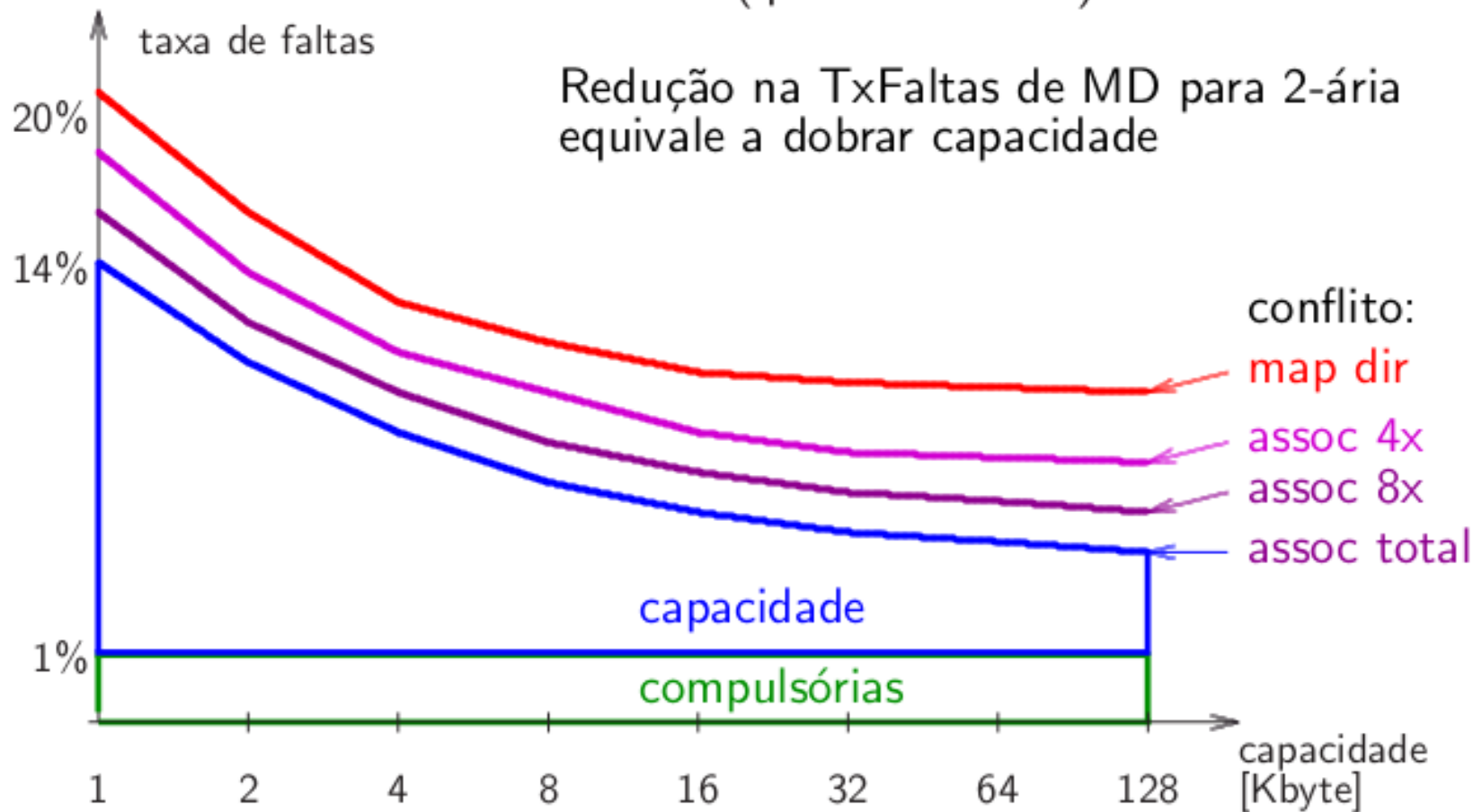
Falta por capacidade

problema: cache não tem capacidade para acomodar todos blocos acessados por programa



Falta compulsória

problema: primeiro acesso a um bloco **sempre** causa uma falta (que é inevitável)



Tipos de faltas e soluções

faltas por conflito

problema: N blocos da mesmo índice (côr) em uso simultâneo, mas cache só acomoda $M < N$ blocos

solução: aumentar associatividade

faltas por capacidade

problema: cache não tem capacidade para acomodar todos blocos acessados por programa

solução: aumentar tamanho da cache

faltas compulsórias

problema: primeiro acesso a um bloco **sempre**

solução: busca antecipada por hardware ou por software

Mais sobre faltas

Que tipos de faltas ocorrem numa cache totalmente associativa de tamanho infinito?

faltas compulsórias porque deve trazer cada bloco para a cache

Além destes, que tipos de faltas ocorrem numa cache totalmente associativa, de tamanho finito?

faltas por capacidade porque programa pode usar mais blocos do que a cache comporta

Além destes, que tipos de faltas ocorrem numa cache associativa por conjunto ou com mapeamento direto?

faltas por conflito porque dois endereços $\neq$ s mapeiam no mesmo bloco da cache (mesmo índice)

Conflict misses are misses that would not occur if the cache was fully associative and had LRU replacement [jouppi90]

Política de troca de blocos

Em **cache associativas**, depois de uma falta de leitura, se não há blocos vazios, qual bloco deve ser expurgado da cache?

bloco usado menos recentemente?

Parece ser o melhor, mas pode ser difícil de implementar

escolher aleatoriamente?

É fácil de implementar, mas como é o desempenho?

TxFaltas Cache Assoc Binária

capac	LRU	aleatória
16KB	5.2%	5.7%
64KB	1.9%	2.0%
256KB	1.15%	1.17%

least recently used = LRU

Implementação:

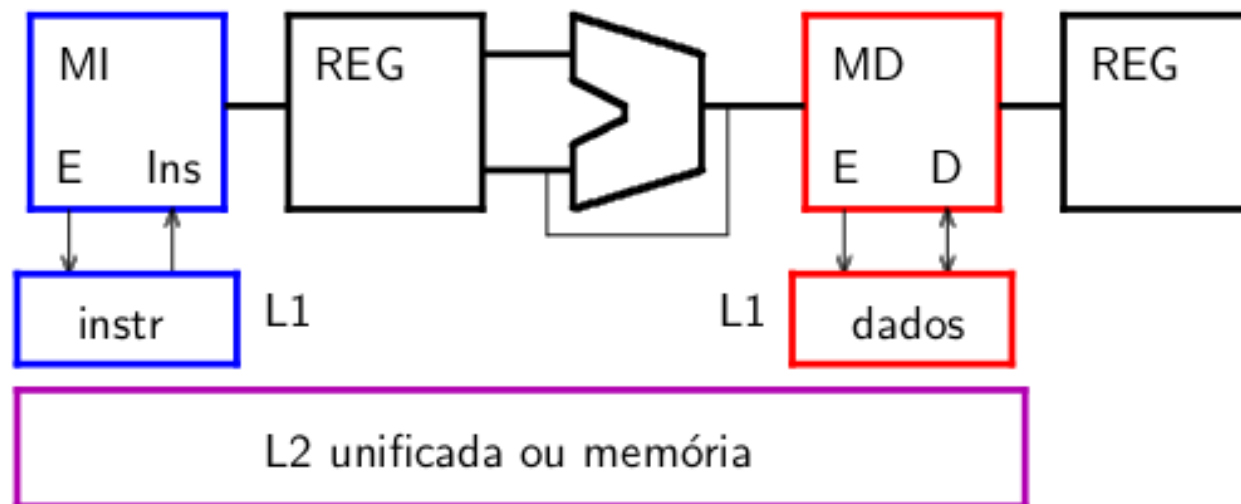
LRU 2-way: bit aponta para LRU

LRU 4-way: 3 bits, hierárquico

Aleatória: (a) somatório de alguns bits do endereço; (b) registrador de deslocamento com realimentação

Caches e pipelines

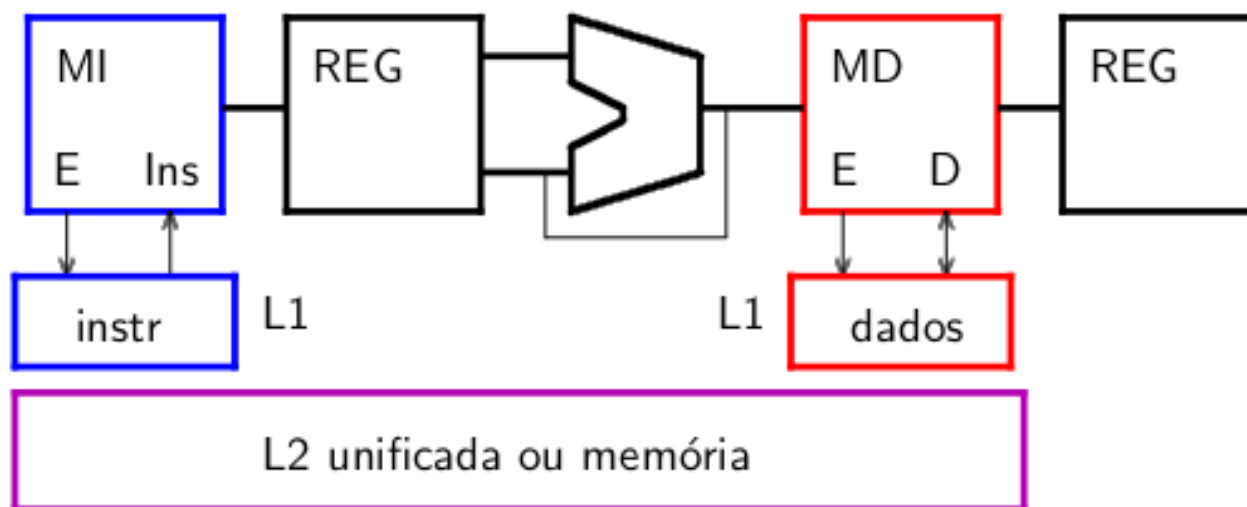
Tempo de Acerto é crucial porque limita o ciclo da CPU
Se muito longo → mais de um ciclo para acessar a cache nos acertos



Há duas portas para memória para eliminar risco estrutural;
2 caches separadas, uma para instruções, uma para dados porque
localidade no acesso aos dados é diferente do acesso a instruções

Caches e pipelines

Acertos: se encontrou inst/dado na cache,
entrega para unidade funcional (RegInstr ou MEM) e prossegue



Faltas: segura execução até que memória entregue bloco faltante

instrução: segura busca, outras instruções prosseguem

dado: deve parar a busca até que memória entregue dado

Desempenho

Equação do desempenho: $\frac{\text{segs}}{\text{prog}} = \frac{\text{instr}}{\text{prog}} \times \frac{\text{ciclos}}{\text{instr}} \times \frac{\text{segs}}{\text{ciclo}}$

Até agora, CPI com tempo de acesso à memória constante:

$$\text{CPI}_{\text{real}} = \text{CPI}_{\text{ideal}} + \text{ciclos esperando pela memória}$$

CPI_{real} depende do **Tempo Médio de Acesso à Memória**

$$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

objetivo de projeto: **minimizar TMAM para dados e instruções**

cuidado: melhorar um termo pode piorar outros e aumentar TMAM

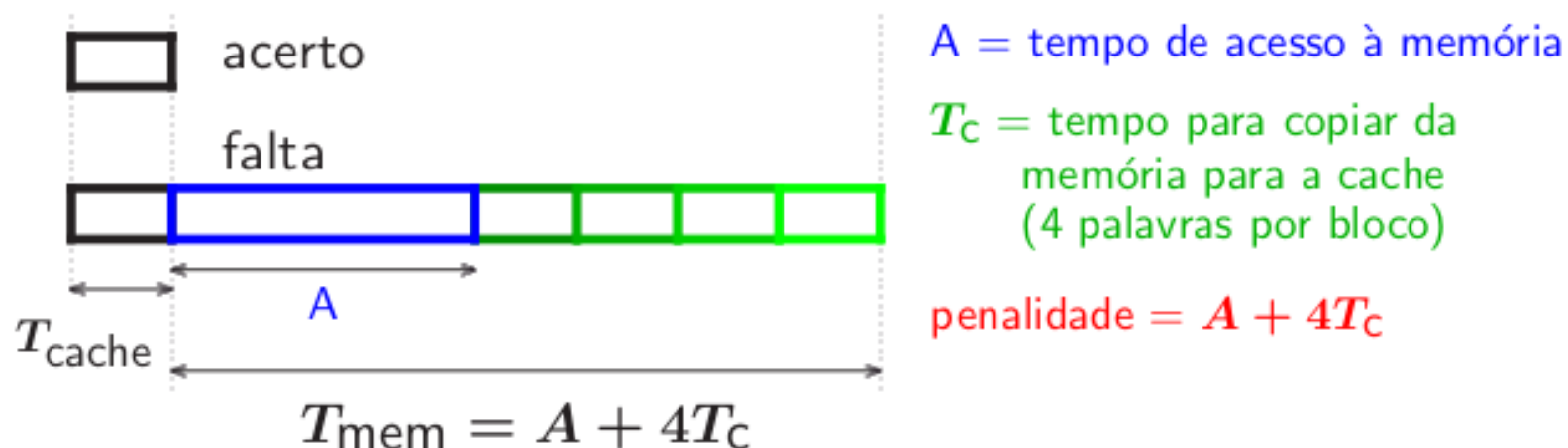
.

AMAT = Average Memory Access Time

Desempenho

Desempenho do sistema com cache

$$TMAM = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$



Compilador GCC com memória perfeita tem $\text{CPI} = 1.2$ (dependências), 11% das referências causam faltas que custam 10 ciclos cada.

$$\begin{aligned} \text{CPI}_{\text{real}} &= (\text{CPI}_{\text{ideal}} + \text{faltas} \times \text{penalidade}) \\ &= (1.2 + 0.11 \cdot 10) = 2.3 \end{aligned}$$

Desempenho

Minimizar tempo de acerto

$$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

- Tempo de acesso é proporcional à capacidade da memória
→ caches pequenas
- Mapeamento Associativo é **mais lento** do que mapeamento direto
acesso às etiquetas ; **comparação** ; **propagar através do MUX**
- Segmentar o acesso à cache (aumenta vazão, mantém latência $\approx$)
endereçamento ; acesso aos dados ; transferência 2-3 estágios

adiciona estágios ao pipeline da CPU:

2-3 est em BUSCA + 2-3 em MEM

Desempenho

Minimizar taxa de faltas

$$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

- Minimizar a taxa de faltas, ajustando para os 3Cs
 - * aumentar capacidade tempo de acesso ↗
 $\text{capacidade} = \text{num_blocos} \times \text{tam_bloco} \times \text{tam_palavra}$
 - * aumentar associatividade tempo de acesso ↗
 $\text{capacidade} = (\text{n_blocos} \times \text{associativ}) \times \text{t_bloco} \times \text{t_palavra}$
 - * busca antecipada por hardware ou por software
cuidado para não desalojar blocos úteis poluição
traz blocos e armazena em buffer específico

$$\begin{aligned} \text{capacidade MD} &= \text{num_blocos} \times \text{tam_bloco} \times \text{tam_palavra} \\ [\text{byte}] &= [\text{bloco}] \times [\text{pal/bloco}] \times [\text{byte/pal}] \end{aligned}$$

Desempenho

Minimizar penalidade i

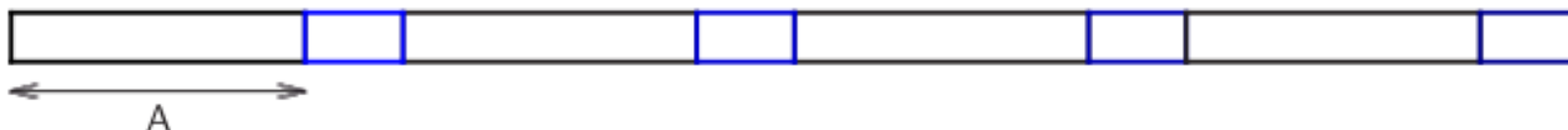
$$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

- Minimizar o tempo de transferência com barramento mais eficiente

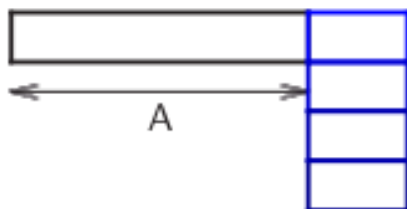
barramento largo (e caro);
cache organizada em bancos;
barramento CPU-cache com transações;

cache de vítimas: cache associativa
pequena (4-8 blcs) mantém blocos
expurgados da L1
blocos na CdV tem 2a chance

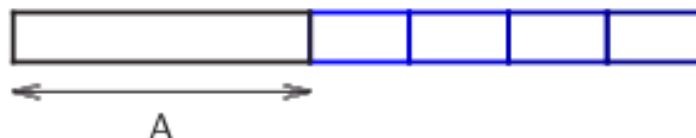
barramento estreito $4(A+1)$



barramento largo $(A+1)$



memória entrelaçada $(A+4)$



Desempenho

Minimizar penalidade ii

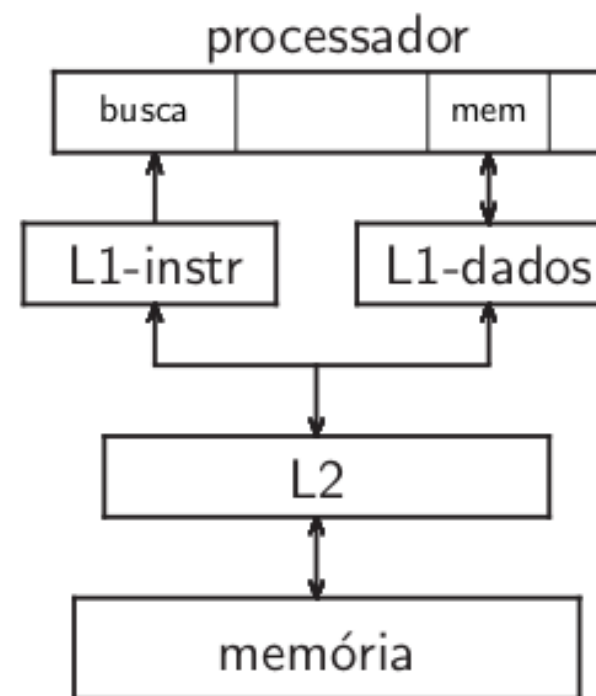
$$TMAM = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

Minimizar tempo de acerto
no nível inferior (L2 ou memória)

$$Tx\text{FaltasLocal } L_n = \frac{\text{faltas-}L_n}{\text{refs-}L_n}$$

$$Tx\text{FaltasGlobal} = \frac{\text{faltas-L1,2}}{\text{refs-CPU}}$$

Taxa de faltas na L2 é alta porque
L1 filtra referências com boa localidade,
sobram para L2 refs com localidade ruim



$$T_{\text{mam}} = T_{L1} + F_{L1} \cdot [T_{L2} + F_{L2} \cdot T_m]$$

Desempenho

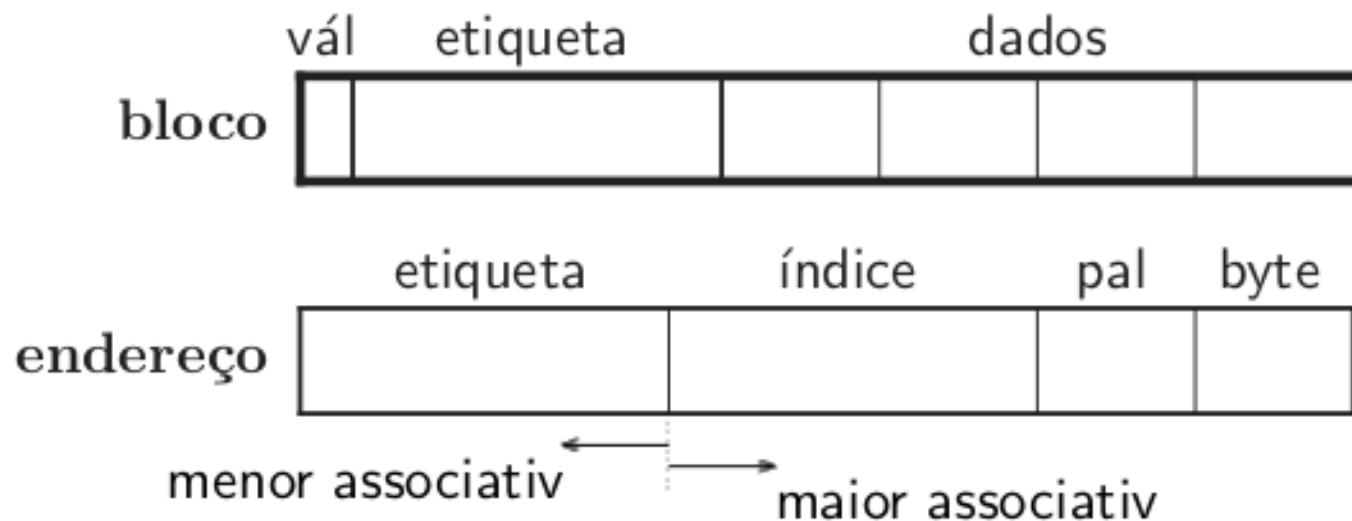
Minimizar penalidade iii

$$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$$

- **Minimizar tempo de carga:** preenchimento do bloco é demorado
- ...mas não precisa esperar até que **todo** bloco seja preenchido
- **Early restart** – assim que palavra requisitada chegar da memória, entrega ao processador, que continua a executar (instruções)
- **Critical word first** – busca palavra requisitada primeiro e a entrega ao processador assim que chegar da memória (CPU continua)
 - * processador requisita **p2**:

p0	p1	p2	p3
----	----	-----------	----
 - * **p2** é entregue, e bloco é preenchido com **p2** → p3 → p0 → p1
 - * localidade espacial indica que próximo acesso será na palavra seguinte, que pode não ter chegado ainda

Organização



$\text{capacidade} = \text{num_blocos} \times \text{tam_bloco} \times \text{tam_palavra} \times \text{associativ}$

$|\text{índice}| = \log_2 \text{num_blocos}$ altura da matriz

$|\text{pal}| = \log_2 \text{tam_}$

Resumindo

Hashing para mapear espaço grande em armazenador pequeno
se largura aumenta ($\text{tamBloco} > 1$) então altura diminui

Três formas de hashing:

- * mapeamento direto $\rightarrow \text{posição} = \text{ender} \% \text{num_blocos}$
- * totalmente associativo $\rightarrow \text{posição} = \forall \text{ no conjunto (índice)}$
- * associativo por conjuntos $\rightarrow \text{posição} = \text{ender} \% \text{num_conj}$

Três categorias de faltas:

- * por capacidade $\rightarrow$ cache maior
- * por conflitos $\rightarrow$ associatividade
- * compulsórias $\rightarrow$ busca antecipada (?)

$\text{TMAM} = \text{TempoDeAcerto} + (\text{TaxaDeFaltas} \times \text{PenalidadePorFalta})$
melhorar um termo geralmente piora outro/s

Créditos

Figuras:

- Patterson&Hennessy 2018, 2004, Morgan Kaufmann
- Roberto Hexsel, 2010, UFPR
- Ulrich Drepper, 2007, Red Hat



Attribution-NonCommercial-ShareAlike 4.0
International

CI1212

Arquitetura de Computadores



Prof. Eduardo Todt

todt@inf.ufpr.br

Período Especial 2020

